```
$ ros2
usage: ros2 [-h] Call `ros2 <command> -h` for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

optional arguments:
  -h, --help            show this help message and exit

Commands:
  action     Various action related sub-commands
  component  Various component related sub-commands
  daemon     Various daemon related sub-commands
  doctor     Check ROS setup and other potential issues
  interface  Show information about ROS interfaces
  launch     Run a launch file
  lifecycle  Various lifecycle related sub-commands
  msg        Various msg related sub-commands
  multicast  Various multicast related sub-commands
  node       Various node related sub-commands
  param      Various param related sub-commands
  pkg        Various package related sub-commands
  run        Run a package specific executable
  security   Various security related sub-commands
  service    Various service related sub-commands
  srv        Various srv related sub-commands
  topic      Various topic related sub-commands
  wtf        Use `wtf` as alias to `doctor`

  Call `ros2 <command> -h` for more detailed usage.
```

From this one command you can figure out what every single ROS 2 CLI program does and how to use it. The ROS 2 CLI has a syntax just like most languages. All ROS CLI commands start with `ros2`, followed by a command. After the command any number of other things can come; you can append `--help` or `-h` to see the documentation and find out what arguments any of the commands are expecting. The rest of this section just walks through each of the commands one by one.

Writing commands using the command line is tricky and error prone. There are a couple of tools you can use to make the process much smoother. The first is the `TAB` key, which attempts to auto complete whatever you type. It can't read your mind, but for common command combinations you usually only need to type the first one or two letters. Another tool is the up arrow key. When you use the command line sometimes you mistype a command, or need to rerun a command. Pressing the up key will cycle through the previous commands which you can modify and rerun as needed.

Running Your First ROS Program

Let's get started with our first ROS CLI command. The first command we'll visit is `run`. Let's start by looking at the documentation for the `run` command:

```
$ ros2 run
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...
ros2 run: error: the following arguments are required: package_name,
executable_name, argv
```

To get more complete information about a ROS 2 command, simply ask the command for help by adding `--help` to the command. Let's try that again:

```
$ ros2 run --help
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...

Run a package specific executable

positional arguments:
  package_name     Name of the ROS package
  executable_name  Name of the executable
  argv             Pass arbitrary arguments to the executable

optional arguments:
  -h, --help       show this help message and exit
  --prefix PREFIX  Prefix command, which should go before the executable.
                   Command must be wrapped in quotes if it contains spaces
                   (e.g. --prefix 'gdb -ex run --args').
```

We can see that `ros2 run` is the command to, "Run a package specific executable." In ROS 2 collections of ROS software are gathered into logical units called `packages`. Each package contains all of the source code for the package as a variety of other data that tells ROS how to build and compile the package and the names of all the programs, also called `executables`, that can be found in the package. The line below the description then gives the *positional arguments* for the package. Positional arguments are the words and values that come after `ros2` and the command you run. In this case the syntax for the command sentence we want to write is as follows:

```
ros2 run <package name> <program/executable name> <args>
```

There is one piece of missing information here. What is this `argv` that the command is asking for? The `argv` element is programmer short hand for variable arguments, and it simply means, "some number of additional arguments that are determined by the executable". It is worth noting that a program can have zero arguments and you can just leave it blank. This is actually how a lot of programs work. For example, say we had a package called *math*, and an executable

called *add* that takes in two numbers and returns the result. In this case *argv* would be the two numbers to add. The final command would look like:

```
ros2 run math add 1 2
```

Finally, below the positional arguments we have *optional arguments*. You don't need to included them, unless you need to.

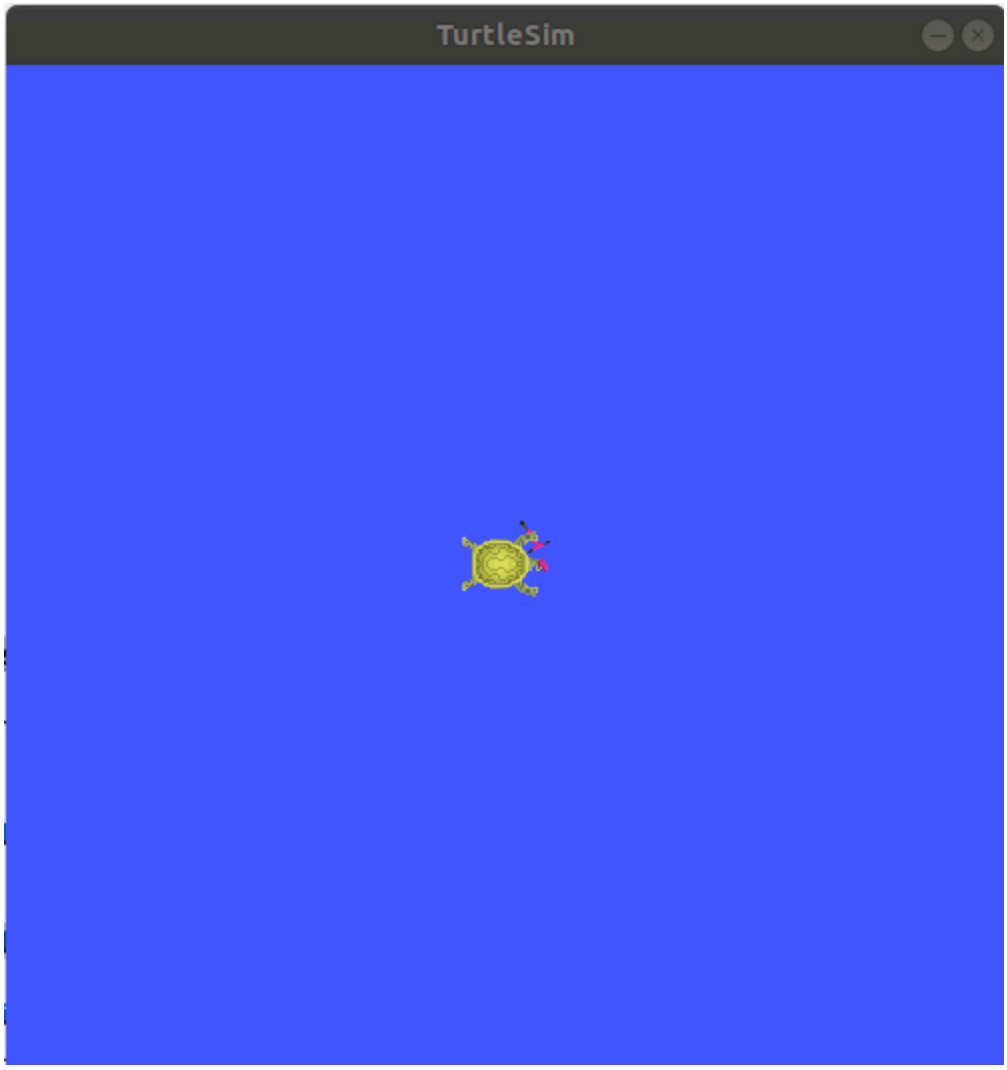
Now that we've looked into our help file let's run our first ROS program. For these tutorials we're going to use a package called `turtlesim`, and the program we want to run is `turtlesim_node`. Let's run this program (remember your tab complete!). Your command should look like the following:

```
ros2 run turtlesim turtlesim_node
```

If everything goes smoothly you should see the following:

```
[INFO] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445],
theta=[0.000000]
```

A window should also pop up with a cute little turtle that looks like the one below:

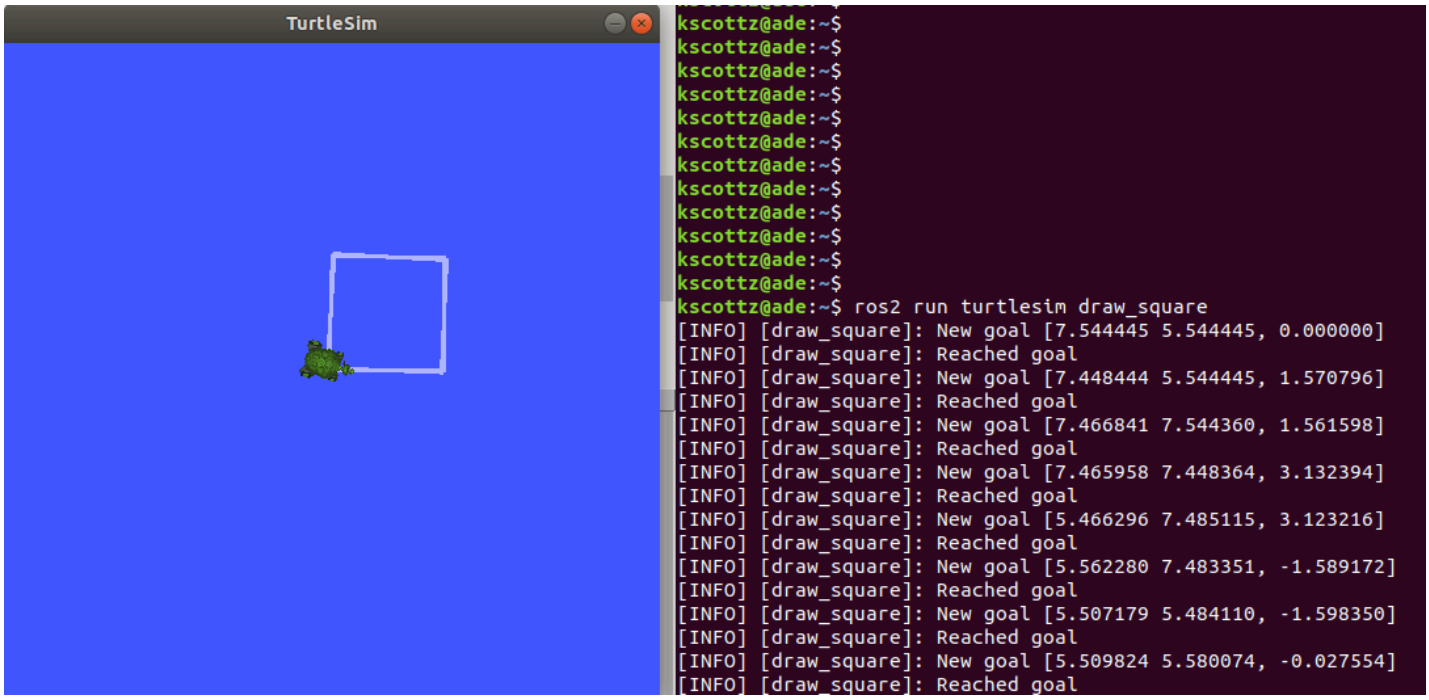


The real power in ROS isn't that it can run a program, it is that it can run lots of programs all at the same time, all talking together to control a robot, or multiple robots, all working together. To illustrate this let's run a second ROS program that makes our little turtle move around.

To do this we'll first open a new terminal (using `CTRL-SHIFT-T`). Next we'll tell that terminal that we want to use ROS Eloquent by using the `source` command. Finally, we'll run another program in the `turtlesim` package to draw a square. See if you can find the program yourself (hint: use `TAB`). If everything works you should have typed the following, and the following output should be visible:

```
$ source /opt/ros/eloquent/setup.bash
$ ros2 run turtlesim draw_square
[INFO] [draw_square]: New goal [7.544445 5.544445, 0.000000]
[INFO] [draw_square]: Reached goal
[INFO] [draw_square]: New goal [7.448444 5.544445, 1.570796]
[INFO] [draw_square]: Reached goal
```

Your screen should look roughly like this:



It is worth noting that you can stop any ROS program by pressing the `Ctrl` and `C` keys at the same time in the terminal; we call this `CTRL-C` (note that `CTRL-SHIFT-C` and `CTRL-SHIFT-V` are responsible for copy and paste in a Linux terminal). Feel free to try it out. Start and stop the programs, and then restart them before moving on.

ROS Topics

We now have two ROS 2 programs running from the `turtlesim` package. There is `turtle_node` that opens our turtle simulation, and `draw_square` that makes the turtle in `turtle_node` move around. How are these two programs communicating?

ROS programs, also called *nodes*, communicate over *topics* on the ROS *message bus*. ROS *topics* use namespaces to distinguish themselves. For example, in a vehicle running ROS, the positions of each wheel may be organized as follows:

```
/wheels/front/driver/velocity
/wheels/front/passenger/velocity
/wheels/rear/driver/velocity
/wheels/rear/passenger/velocity
```

The key thing to realize about topics is that the data they contain is dynamic, meaning it changes constantly. In our vehicle example the velocity of each wheel might be measured one thousand times a second or more. Since the data in a ROS topic is constantly changing, an important distinction for a topic is whether the topic is "creating" or as we like to say in ROS `publishing`, or if it is reading the data, what we call `subscribing` to the topic. Many ROS

nodes subscribe to one set of topics, process that input data, and then publish to another set of topics.

Let's return to our turtlesim example and see if we can use the ROS CLI to understand the topics, publishers, and subscribers. To see sub commands and syntax for the `topic` command, we'll run: `ros2 topic --help`.

This command outputs the following:

```
$ ros2 topic --help
usage: ros2 topic [-h] [--include-hidden-topics]
                  Call `ros2 topic <command> -h` for more detailed usage. ...

Various topic related sub-commands

optional arguments:
  -h, --help            show this help message and exit
  --include-hidden-topics
                        Consider hidden topics as well

Commands:
  bw     Display bandwidth used by topic
  delay  Display delay of topic from timestamp in header
  echo   Output messages from a topic
  find   Output a list of available topics of a given type
  hz     Print the average publishing rate to screen
  info   Print information about a topic
  list   Output a list of available topics
  pub    Publish a message to a topic
  type   Print a topic's type

  Call `ros2 topic <command> -h` for more detailed usage.
```

There are quite a few sub commands; we won't discuss all of them, but let's look closely at a few. Sub commands have their own help command. Why don't we examine the `list` command. Repeating our command pattern let's try running `ros2 topic list --help`.

```
usage: ros2 topic list [-h] [--spin-time SPIN_TIME] [-t] [-c]
                       [--include-hidden-topics]

Output a list of available topics

optional arguments:
  -h, --help            show this help message and exit
  --spin-time SPIN_TIME
                        Spin time in seconds to wait for discovery (only
                        applies when not using an already running daemon)
  -t, --show-types      Additionally show the topic type
  -c, --count-topics    Only display the number of topics discovered
  --include-hidden-topics
                        Consider hidden topics as well
```

As indicated at the top of this command help file, `ros2 topic list` will "Output a list of available topics." There appears to be a variety of *optional* arguments that we don't need to include if we don't want to. However, the `-t, --show-types` line looks interesting. It is worth noting that command arguments, sometimes called flags, can have two types. A short form indicated with a single dash ("-"), and a long form indicated by a double dash ("--"). Don't worry, despite looking different both versions of the argument do the same thing. Let's try running this command, sub command pair with the `-show-types` argument.

```
$ ros2 topic list --show-types
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

On the left hand side we see all of the ROS topics running on the system, each starting with `/`. We can see that most of them are gathered in the `/turtle1/` group. This group defines all the inputs and outputs of the little turtle on our screen. The words in brackets (`[]`) to the right of the topic names define the messages used on the topic. Our car wheel example was simple, we were only publishing velocity, but ROS allows you to publish more complex data structures that are defined by a *message type*. When we added the `--show-types` flag we told the command to include this information. We'll dig into messages in detail a bit later.

One of the more commonly used topic sub commands is `info`. Unsurprisingly, `info` provides info about a topic. Let's peek at its help file using `ros2 topic info --help`

```
$ ros2 topic info --help
usage: ros2 topic info [-h] topic_name

Print information about a topic

positional arguments:
  topic_name  Name of the ROS topic to get info (e.g. '/chatter')

optional arguments:
  -h, --help  show this help message and exit
```

That seems pretty straight forward. Let's give it a go by running it on `/turtle1/pose`

```
$ ros2 topic info /turtle1/pose
Type: turtlesim/msg/Pose
Publisher count: 1
Subscriber count: 1
```

What does this command tell us? First it tells us the *message type* for the `/turtle1/pose` topic, which is `/turtlesim/msg/Pose`. From this we can determine that the message type comes from the *turtlesim* package, and its type is `Pose`. ROS messages have a predefined message type that can be shared by different programming languages and between different nodes. We can also see that this topic has a single publisher, that is to say a single node generating data on the topic. The topic also has a single subscriber, also called a listener, who is processing the incoming pose data.

If we only wanted to know the message type of a topic there is a sub command just for that called, `type`. Let's take a look at its help file and its result:

```
$ ros2 topic type --help
usage: ros2 topic type [-h] topic_name

Print a topic's type

positional arguments:
  topic_name  Name of the ROS topic to get type (e.g. '/chatter')

optional arguments:
  -h, --help  show this help message and exit
kscottz@kscottz-ratnest:~/Code/ros2multirobotbook$ ros2 topic type /turtle1/pose
turtlesim/msg/Pose
```

While it is not part of the `topic` command it is worthwhile for us to jump ahead briefly and look at one particular command, sub command pair, namely the `interface` command and the `show` sub command. This sub command will print all the information related to a message type so you can better understand the data being moved over a topic. In the previous example we

saw that the `topic type` sub command told us the `/turtle1/pose` topic has a type `turtlesim/msg/Pose`. But what does `turtlesim/msg/Pose` data look like? We can look at the data structure transferred by this topic by running the `ros2 interface show` sub command and giving the message type name as an input. Let's look at the help for this sub command and its output:

```
$ ros2 interface show --help
usage: ros2 interface show [-h] type

Output the interface definition

positional arguments:
  type        Show an interface definition (e.g. "std_msgs/msg/String")

optional arguments:
  -h, --help  show this help message and exit

$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

We can see the values `x` and `y` which are the position coordinates of our turtle, and that they are of type `float32`. `theta` is the direction the head is pointing. The next two values, `linear_velocity` and `angular_velocity`, are how fast the turtle is moving and how quickly it is turning, respectively. To summarize, this message tells us where a turtle is on the screen, where it is headed, and how fast it is moving or rotating.

Now that we know what ROS topics are on our simple turtlesim, and their message types, we can dig in and find out more about how everything works. If we look back at our topic sub commands, we can see a sub command called `echo`. Echo is computer jargon that means "repeat" something. If you echo a topic it means you want the CLI to repeat what's on a topic. Let's look at the `echo` sub command's help text:

```
$ ros2 topic echo --help
usage: ros2 topic echo [-h]
                       [--qos-profile
{system_default,sensor_data,services_default,parameters,parameter_events,action_st
atus_default}]
                       [--qos-reliability {system_default,reliable,best_effort}]
                       [--qos-durability
{system_default,transient_local,volatile}]
                       [--csv] [--full-length]
                       [--truncate-length TRUNCATE_LENGTH] [--no-arr]
                       [--no-str]
                       topic_name [message_type]

Output messages from a topic

positional arguments:
  topic_name            Name of the ROS topic to listen to (e.g. '/chatter')
  message_type          Type of the ROS message (e.g. 'std_msgs/String')

optional arguments:
  -h, --help            show this help message and exit
  --qos-profile
{system_default,sensor_data,services_default,parameters,parameter_events,action_st
atus_default}
                        Quality of service preset profile to subscribe with
                        (default: sensor_data)
  --qos-reliability {system_default,reliable,best_effort}
                        Quality of service reliability setting to subscribe
                        with (overrides reliability value of --qos-profile
                        option, default: best_effort)
  --qos-durability {system_default,transient_local,volatile}
                        Quality of service durability setting to subscribe
                        with (overrides durability value of --qos-profile
                        option, default: volatile)
  --csv                 Output all recursive fields separated by commas (e.g.
                        for plotting)
  --full-length, -f     Output all elements for arrays, bytes, and string with
                        a length > '--truncate-length', by default they are
                        truncated after '--truncate-length' elements with
                        '...''
  --truncate-length TRUNCATE_LENGTH, -l TRUNCATE_LENGTH
                        The length to truncate arrays, bytes, and string to
                        (default: 128)
  --no-arr              Don't print array fields of messages
  --no-str              Don't print string fields of messages
```

Wow, that's a lot of features. The top of the help files says that this CLI program "output[s] messages from a topic." As we scan the positional arguments we see one required argument, a topic name, and an optional message type. We know the message type is optional because it has square brackets (`[]`) around it. Let's give the simple case a whirl before we address some of the optional elements. Two things to keep in mind: first is that topics are long and easy to